INSTITUTO UNIVERSITÁRIO DE LISBOA

Advanced Computing

Introduction to MPI

Ricardo Fonseca

ricardo.fonseca@iscte-iul.pt

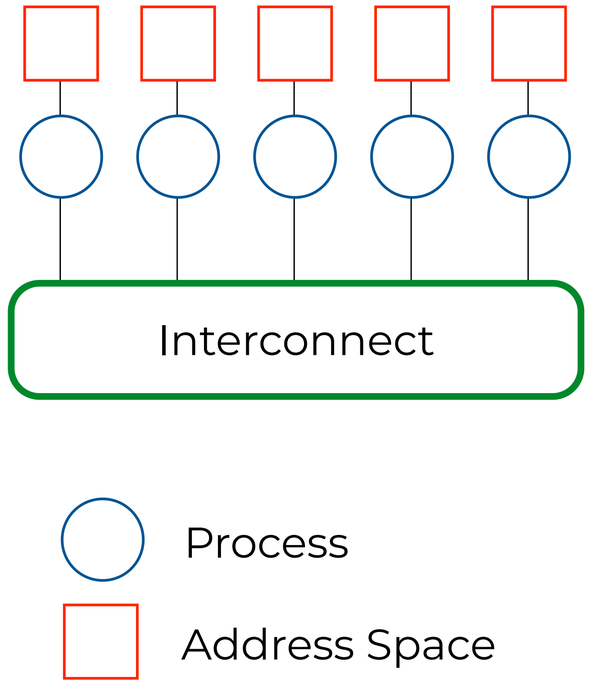
Introduction to MPI

- **Introduction**
  - Message passing model
  - Problem decomposition
  - MPI - message passing interface
- **Programming model**
  - Basic MPI concepts
  - Compiling and running
- **Point to point communication**
  - Blocking messages
  - Datatypes, wildcards and return status
- **Non-blocking communication**
  - Operation and message requests
  - Communication issues
- **Overview**

Introduction

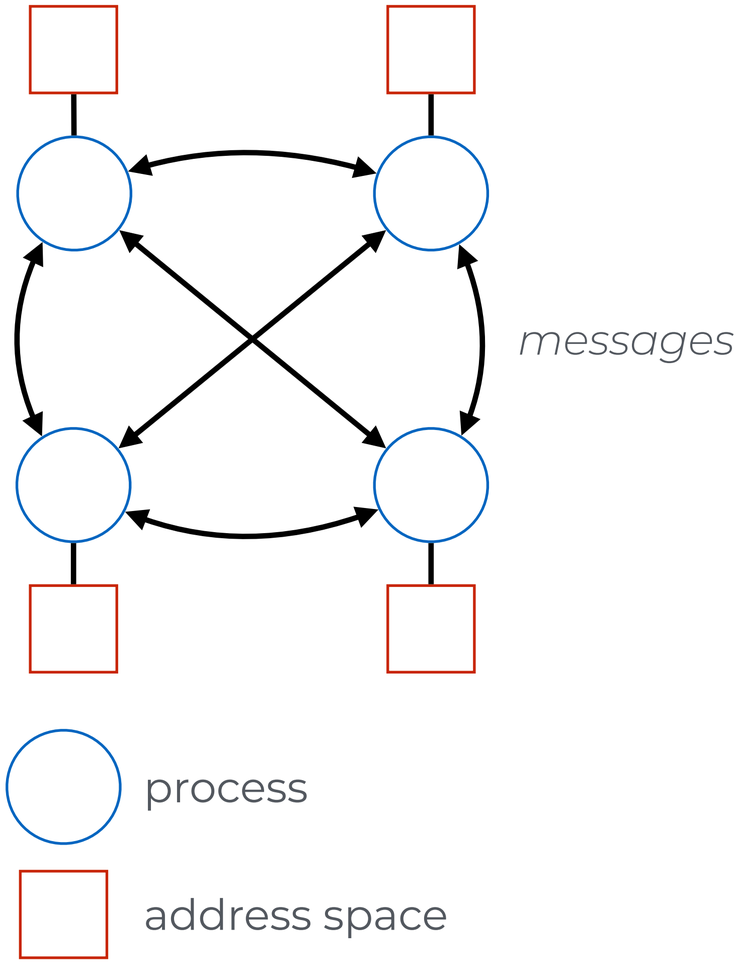
Distributed memory systems

- **Distributed memory systems use separate address spaces for each processor**
  - Processing elements only have direct access to local data
- **Data sharing is harder**
  - Data must be manually decomposed
- **Tasks are processes**
  - More complex, since they do not share context
  - Higher startup/finalize overhead
  - More suited to coarse-grain parallelism
- **Interconnect / Synchronization issues**
  - Interconnect performance limits scalability
  - Synchronization overhead is larger



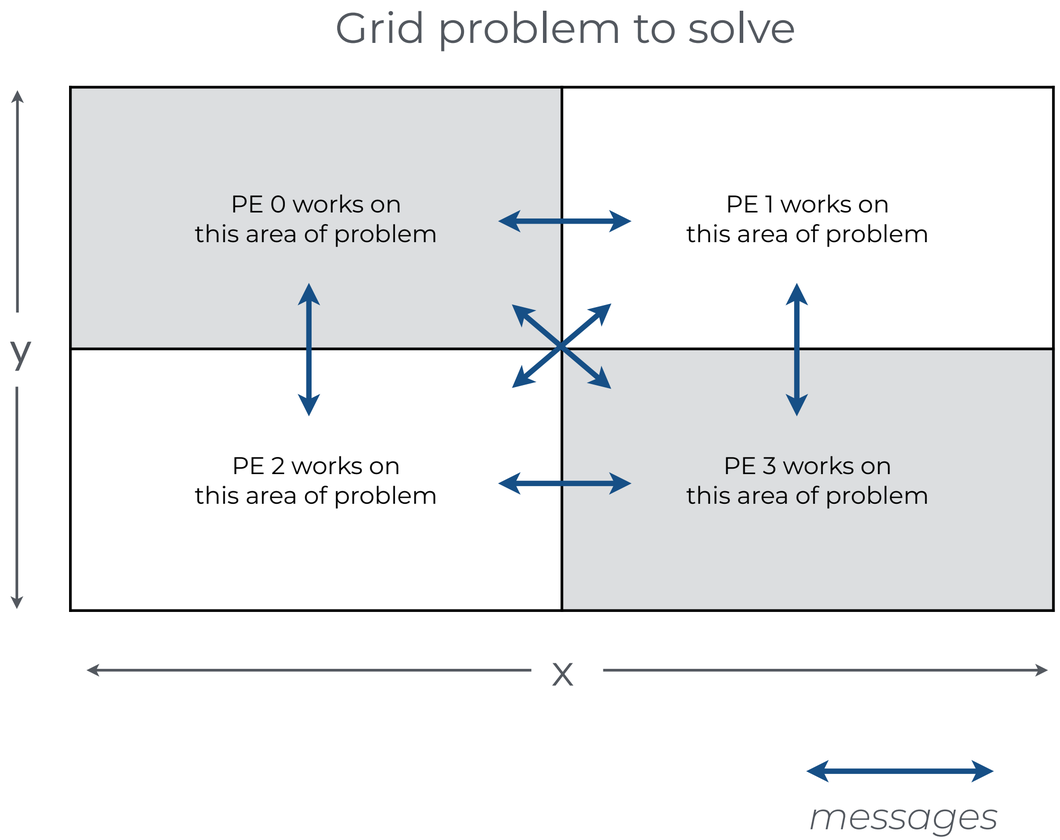
Message passing model

- **Message Passing is the main programming model for distributed memory systems**
  - Every processor executes an independent process using disjoint address spaces (no shared data)
  - Accessing data on a remote processor requires exchanging a message
  - All communication between processes is done cooperatively, through subroutine calls
- **Message passing has succeeded because**
  - Maps well to a wide range of hardware
  - Parallelism and communication are both explicit
  - Forces the programmer to tackle parallelization from the beginning



Parallel decomposition and Communication

- **The parallelization must be considered from the beginning**
  - The programmer must consider problem decomposition amongst processes,
  - And explicitly use messages to communicate between them
  - A good parallelization strategy will be key for performance
- **Both data and functional decompositions can be used**
  - Each processing element may perform the same task on different data
  - Each processing element may perform different tasks
- **Example: Spatial domain decomposition**
  - Decompose large grid into smaller domains
  - Each process handles 1 domain
  - After each iteration, processes synchronize and exchange edge values using messages



MPI - Message Passing Interface

- **Message Passing Interface (MPI) is the de facto standard for programming on distributed memory parallel computers**
  - Library of routines that enable message passing applications
  - Interface specification, not a concrete implementation
- **MPI implements a message Passing Model**
  - Each process has its own local address space
  - Processes communicate with each other through messages
- **MPI works with both distributed and shared memory**
  - Processes are logical, not physical
  - Extremely general/portable; even allows for heterogeneous systems

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank, size;

   /* Initialize MPI */
   MPI_Init(&argc, &argv);
   /* Get total number of procs. */
   MPI_Comm_size( MPI_COMM_WORLD, &size );
   /* Get my rank */
   MPI_Comm_rank( MPI_COMM_WORLD, &rank );

   printf("I'm proc. %d of %d\n",rank,size);

   /* Close MPI */
   MPI_Finalize();
}
```

Using MPI on your system

- **MPI implementations are distributed as libraries**
  - Include header files and libraries, compiler wrappers, and tools
  - Standard supports C/C++ and Fortran

- **Install one MPI flavor in your laptop/desktop**
  - OpenMPI - http://www.open-mpi.org/ (we will be using this)
  - MPICH - https://www.mpich.org/
  - Microsoft MPI - https://learn.microsoft.com/en-us/message-passing-interface/

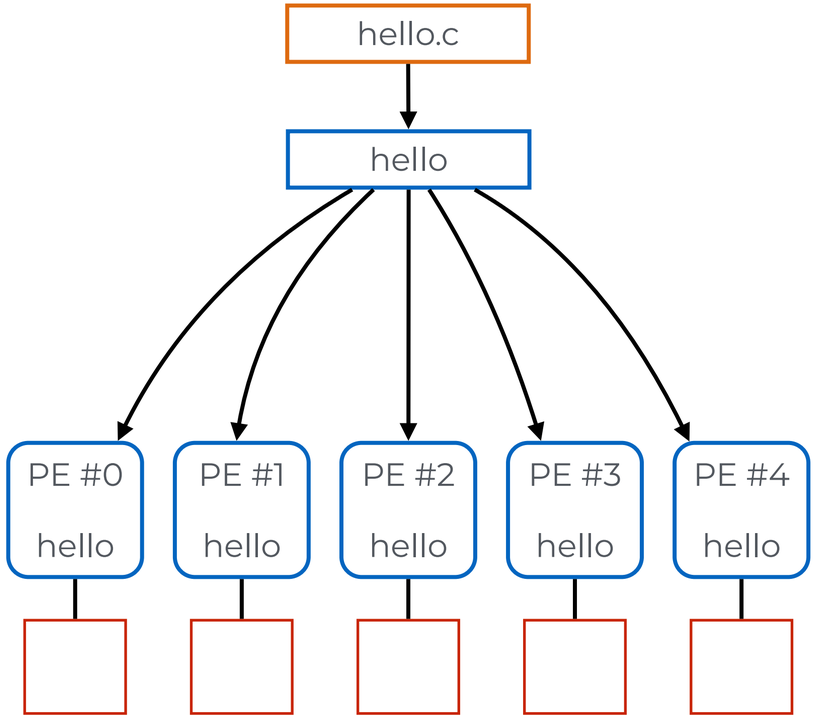
MPICH

- **Develop and Debug locally**
  - You can launch multiple processes in a single machine, emulating a large parallel environment
  - Once finished debugging compile and deploy on the larger system for production runs
  - All MPI libraries are (should be) compatible

MPI Programming model

Programming model

- **MPI programs use a SPMD programming model**
  - Single Program, Multiple Data
  - Only 1 source code is written
  - Code can have conditional execution based on which processor is executing the copy
- **All copies will be launched simultaneously**
  - The system will start the processes on the allocated processing elements
  - Processes will then communicate and sync with each other periodically



Ranks and Communicators

- **In the context of an MPI program**
  - A **communicator** defines a set of tasks and the communication channels between them
- **When the MPI program starts it automatically creates a communicator**
  - This communicator, called `MPI_COMM_WORLD`, includes all tasks launched at startup
  - As we will see later, it will be possible to partition our MPI program into multiple communicators
- **Tasks within a communicator have a unique value**
  - The MPI **rank** allows the program to identify the task
  - Rank numbers start with 0

MPI program structure

- **MPI programs must include the `mpi.h` header**
  - Provides interfaces to MPI library calls and constants
- **All MPI programs must start by initializing the MPI environment**
  - The `MPI_Init()` call sets up the communication channels between all tasks
  - Also ensures that all tasks receive the same command line arguments
- **The `MPI_COMM_WORLD` communicator is available after initialization**
  - The `MPI_Comm_size()` allows us to probe the global size of the communicator
  - The process unique id (rank) may be obtained using the `MPI_Comm_rank()` function
- **MPI library should be shutdown gracefully**
  - The `MPI_Finalize()` call ensures proper program finalization

```c
#include <mpi.h>
// …

// Initialization
MPI_Init(&argc, &argv);

MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

// …

// Cleanup
MPI_Finalize();
```

Hello world

Initialize MPI

Get total number of processes

Get local process number

Close MPI

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank, size;

   /* Initialize MPI */
   MPI_Init(&argc, &argv);
   /* Get total number of procs. */
   MPI_Comm_size( MPI_COMM_WORLD, &size );
   /* Get my rank */
   MPI_Comm_rank( MPI_COMM_WORLD, &rank );

   printf("I'm proc. %d of %d\n",rank,size);

   /* Close MPI */
   MPI_Finalize();
}
```

Compiling and running

- **MPI programs can be compiled as any other program that uses an external library**
  - To simplify the process, MPI distributions usually include a “wrapper compiler”
  - This tool sets the appropriate include paths and links the required libraries automatically
- **MPI distributions also include a tool for launching MPI programs**
  - This tool is usually called `mpiexec` or `mpirun`
- **Launching on large scale systems is usually done differently**
  - The queuing system will call `mpiexec` for us

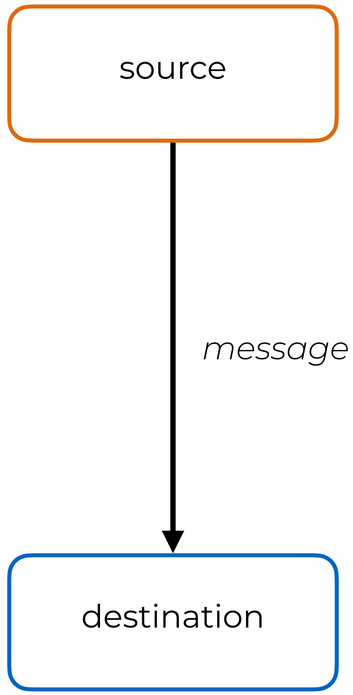
```
$ mpicc hello.c -o hello
```

```
# N is the number of processes
$ mpiexec -np <N> ./hello
I’m proc. 1 of 4
I’m proc. 2 of 4
I’m proc. 3 of 4
I’m proc. 0 of 4
```

Point to point communication

MPI messages

- **As we saw earlier, MPI tasks will communicate between them using messages**
  - This is a 2-sided cooperative process: both the sender and the receiver must explicitly participate in the exchange
- **When sending a message, the sender must call an MPI function to**
  - Identify the destination (rank of destination task)
  - Specify the message size and datatype
  - Specify a message tag that may used to identify this specific message by the destination
- **When receiving a message, the destination must call an MPI function to**
  - Identify the sender of the message (rank of sender task)
  - Specify the datatype and **maximum** message size. It is possible to receive smaller messages.
  - Specify the tag of the incoming message. Messages with different tags will be ignored.
- **Both should use the same communicator**



Basic communications

- **The basic MPI message functions are:**
  - `MPI_Send()` - send message
  - `MPI_Recv()` - receive message
- **These are blocking communications**
  - `MPI_Send()` will only return when the send buffer can be reused (does not guarantee the message has been delivered)
  - `MPI_Recv()` will only return when the receive buffer has been filled with valid data
- **The program will not proceed until the communication has completed**
  - Or, at least on the sender side, it has been buffered elsewhere
  - Risks deadlocks

```
MPI_Send(void* buf,
         int count,
         MPI_Datatype type,
         int dest,
         int tag,
         MPI_Comm comm);

MPI_Recv(void* buf,
         int count,
         MPI_Datatype type,
         int source,
         int tag,
         MPI_Comm comm,
         MPI_Status* status);
```

Example

- **Simple communication example**
  - Rank 0 sends a value to rank 1
  - Rank 1 receives value and prints it

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char* argv[]) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        int value = 42;
        MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else if (rank == 1) {
        int value;
        MPI_Recv(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Rank 1 received %d\n", value);
    }

    MPI_Finalize();
    return 0;
}
```

Datatypes

- **Messages must set which datatype is being communicated**
  - Message count parameter refers to datatype elements, not bytes
- **Avoids having to identify type sizes at runtime**
  - Also allows the library to support heterogeneous systems (on the fly data conversion)
- **MPI supports a large number of built-in datatypes**
  - It is also possible to define custom types (e.g., structures)
- **Using `MPI_BYTE` is also possible**
  - In this case the message is “type agnostic”, i.e., the memory will be mapped directly to the message buffer

MPI Datatype	C
`MPI_CHAR`	`signed char`
`MPI_DOUBLE`	`double`
`MPI_FLOAT`	`float`
`MPI_INT`	`int`
`MPI_LONG`	`long`
`MPI_UNSIGNED`	`unsigned int`

Wildcards

- **As we saw earlier, when posting a `MPI_Recv()` we have to**
  - Identify the message sender
  - Identity the message tag
- **It is also possible to use wildcards for these fields**
  - Using `MPI_ANY_SOURCE` as sender rank will allow us to receive messages from any source
  - Also on the receiving side, `MPI_ANY_TAG` matches any message
- **Unless there is a good reason to do so, do not use wildcards**
  - The program will no longer guarantee that messages will be received in the same order
  - It is best to structure your message pattern to avoid them

Return status

- **`MPI_Recv()` also allows us to get information on the received message**
  - This is returned through the status parameter
  - If this information is not required use `MPI_STATUS_IGNORE`
- **`MPI_Status` is a structure**
  - `status.MPI_TAG` is the tag of the incoming message (useful if `MPI_ANY_TAG` was specified)
  - `status.MPI_SOURCE` is rank of the sender of the incoming message (useful if `MPI_ANY_SOURCE` was specified)
- **The status can also be used to check how many elements were actually received**
  - Remember, the message size may be smaller than the `count` parameter
  - Use `MPI_Get_count(MPI_Status *status, MPI_Datatype datatype, int *count)`

A six-function version of MPI

- **Core MPI functionality can be implemented with only 6 functions**
  - Initialization
  - Partition size and individual rank information
  - Blocking send/receive communications
  - Finalizations

Name	Function
`MPI_Init`	Initialize MPI
`MPI_Comm_Size`	Find out how many processes there are
`MPI_Comm_Rank`	Find out which process I am
`MPI_Send`	Send a message
`MPI_Recv`	Receive a a message
`MPI_Finalize`	Terminate MPI

Non-blocking communication

What's wrong with this picture?

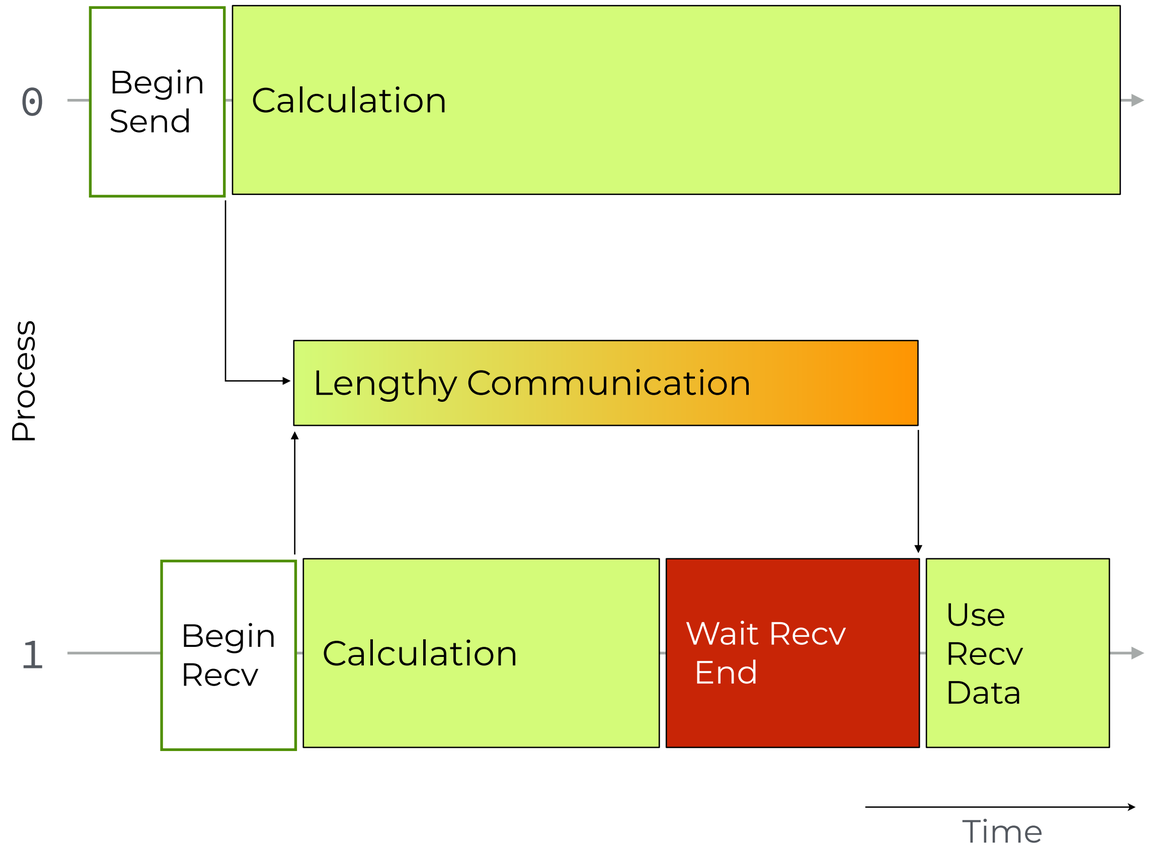
- **The code is meant to swap values between the two ranks**
  - Each ranks sends and receives a value from the other node
- **However, the code will fail**
  - Rank 0 posts a blocking send to rank 1
  - Rank 1 posts a blocking send to rank 0
  - Since no corresponding receives have been posted, the code deadlocks
- **This illustrates the need for non-blocking communications**

✘

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char* argv[]) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int value = rank + 1;

    if (rank == 0) {
        MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Recv(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Swap completed\n");
    } else if (rank == 1) {
        MPI_Send(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        MPI_Recv(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }

    MPI_Finalize();
    return 0;
}
```

Non-blocking communications

- **MPI also supports non-blocking communications**
  - These can be used in both avoiding communication deadlocks and overlapping communication and computation
- **Non-blocking communications calls return immediately**
  - The communication (send or receive) will proceed in the background
  - The message receives a request identifier (MPI_Request)
- **The code will proceed with other activities while the communication proceeds**
  - This may include both computation and additional communications
- **The code must also check for communication completion explicitly**
  - We can check if the message has completed, or explicitly wait for the message to finish
  - It is also possible to cancel an unfinished request



Isend / Irecv

- **The basic MPI non-blocking message functions are:**
  - `MPI_Isend()` - send message
  - `MPI_Irecv()` - receive message
- **The syntax / parameters are very similar to blocking messages**
  - Functions have an additional `request` parameter, which is used to identify the ongoing message
  - Unlike `MPI_recv()`, the `MPI_Irecv()` function does not have a `status` parameter. This will only be available once the message completes
- **The functions will return immediately**
  - The `request` value can then be used to test, wait or cancel the message
- **They can be paired with corresponding blocking calls**
  - e.g. `MPI_Irecv()` ↔ `MPI_Send()`

```
int MPI_Isend( const void *buf,
        int count,
        MPI_Datatype datatype,
        int dest,
        int tag,
        MPI_Comm comm,
        MPI_Request *request );

int MPI_Irecv( void *buf,
        int count,
        MPI_Datatype datatype,
        int source,
        int tag,
        MPI_Comm comm,
        MPI_Request *request );
```

Request operations

- **Once the send/received request is done, the communication proceeds in the background**
  - To control the message we use the corresponding request value
- **`MPI_Wait()` (blocking) waits for the message to complete**
  - If applicable, the status variable will have information regarding the message
  - Can be ignored using `MPI_STATUS_IGNORE`
- **`MPI_Test()` (non-blocking) checks if the message has completed**
  - Result is returned immediately in the flag parameter
  - If the message has completed, returns information in status parameter
- **`MPI_Cancel()` (non-blocking) cancels the request**
  - You will still need to call `MPI_Wait()` to clear the request, but the function will return immediately

```
int MPI_Wait(MPI_Request *request,
    MPI_Status *status);

int MPI_Test(MPI_Request *request,
    int *flag,
    MPI_Status *status);

int MPI_Cancel(MPI_Request *request);
```

Non-blocking communication example

- **The program was rewritten to use non-blocking communications**
  - Use two separate buffers for the send and receive messages, a and b
  - The receive message is non-blocking
  - The send message is blocking, but there's no deadlock
  - After starting communication the program will wait for receive message completion
- **This version overlaps send and receive messages**
  - If the network supports it, it would use full-duplex communication

✔

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char* argv[]) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int a, b;
    MPI_Request request;

    if (rank == 0) {
        MPI_Irecv( &b, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &request );
        MPI_Send(  &a, 1, MPI_INT, 1, 0, MPI_COMM_WORLD );
    } else if (rank == 1) {
        MPI_Irecv( &b, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &request );
        MPI_Send(  &a, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
    }

    MPI_Wait( request, MPI_STATUS_IGNORE );
    MPI_Finalize();
    return 0;
}
```

Non-blocking communication issues

- **The program must not modify the send / receive buffers while the communication is taking place**
  - Otherwise the results are undefined

- **Obvious concerns**
  - Must not modify the buffer between `Isend()` and the corresponding `Wait()`
  - Must not look at or modify the buffer between `Irecv()` and the corresponding `Wait()`
  - Must not have two pending `Irecv()` for the same buffer

- **Less obvious**
  - Must not have two pending `Isend()` for the same buffer
  - MPI is allowed to modify the buffer contents during send

✘

```
// Read before wait
MPI_Irecv( a, 100, MPI_INT, 1, 0, MPI_COMM_WORLD, &request );
printf("a[0] = %d\n", a[0]);
MPI_Wait( request, MPI_STATUS_IGNORE );

// Write before wait
MPI_Isecv( b, 100, MPI_INT, 1, 0, MPI_COMM_WORLD, &request );
b[0] = 123;
MPI_Wait( request, MPI_STATUS_IGNORE );

// 2 receives for the same buffer
MPI_Irecv( a, 100, MPI_INT, 1, 0, MPI_COMM_WORLD, &req1 );
MPI_Irecv( a, 100, MPI_INT, 2, 0, MPI_COMM_WORLD, &req2 );

MPI_Wait( req1, MPI_STATUS_IGNORE );
MPI_Wait( req2, MPI_STATUS_IGNORE );
```

Overview

Overview

- **MPI provides a standard/portable toolset to program for distributed memory parallel computers**
  - Parallelism and communication are both explicit
  - Forces the programmer to tackle parallelization from the beginning
- **MPI implements a message passing model**
  - Provides functions for communication between parallel tasks
  - Basic communication is 2-sided: both sender and destination tasks must call MPI
  - A complete MPI program can use as little as 6 MPI functions
- **MPI point to point messages represent the core of the library**
  - Messages are described by source/destination task, datatype, size (of datatype elements) and tags
  - Incoming messages may be selected based on source and/or tag
  - Messages can be blocking or non-blocking, allowing us to overlap with computation or other messages if needed